Measuring Validator Utility in Generate–Verify–Repair NetOps Pipelines: a Confirmatory Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory study (AC-2026-03) to evaluate whether a multidimensional validator metric suite predicts end-to-end agentic NetOps success better than a single verifier pass rate. Using a synthetic intent→configuration generator and a hosted generate–validate–repair (GVR) adapter under shared_initial_candidate semantics, we ran 200 paired tasks (one shared initial generation per task reused for baseline and guarded). The deterministic validator (deterministic_netops_validator_v5, v5.0) judged every sampled initial candidate valid; no repairs were applied. Baseline = 200/200, guarded = 200/200, paired difference = 0.00 (bootstrap 95% CI [0.00, 0.00], exact McNemar two-sided p = 1.0; bootstrap_seed = 1729, resamples = 10000). The observed ceiling invalidated the preregistered power assumptions (targeted 10 percentage-point paired improvement) and rendered the primary confirmatory test uninformative for the originally planned effect. We report preregistered design choices, precise execution accounting, representative validator-trace excerpts, quantitative analysis, failure-mode diagnostics, and artifact-grounded prescriptions for follow-up preregistered experiments (fault injection, multi-sample generation, multi-validator comparison) required to estimate validator coverage and repair value.**

I. Introduction

Automating NetOps workflows with generative models requires reliable mechanisms to avoid harmful, silent, or wrong-but-plausible actions. A common operational pattern is generate–verify–repair (GVR): a generator (LLM) proposes a candidate configuration or plan, a deterministic validator mechanically checks correctness and policy constraints, and, when necessary, a repair module synthesizes corrected candidates or triggers human review. Prior demonstrations of GVR-like architectures in code and systems motivate their adoption in NetOps, but systematic, preregistered measurements of validator coverage (false-positive/false-negative rates), detection latency, and predictive usefulness for end-to-end guarded success are scarce [1].

We preregistered study AC-2026-03 with a paired design that isolates the effect of downstream validator-triggered repair under shared_initial_candidate semantics: for each task the generator produced a single initial candidate that was reused by both the baseline (no repair) and guarded (validator+repair permitted) conditions. Under these semantics any paired difference can only arise downstream from validator-triggered repair, not from independent generator sampling. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline and the preregistered confirmatory hypothesis (H1) expected that a multidimensional validator-metric suite would explain more variance in end-to-end success than a single verifier pass rate. Our main contribution is a fully preregistered, artifact-archived experiment and an exact, transparent report of a degenerate confirmatory outcome (ceiling) with concrete, artifact-grounded prescriptions for follow-up designs that can measure validator utility.

II. Related Work

The literature relevant to validator utility in GVR-style NetOps pipelines falls into three complementary strands: (i) architectural proposals and demonstrations of GVR or verifier-interposed loops; (ii) controlled benchmark efforts that surface real-world agent failure modes relevant to NetOps; and (iii) system- and survey-level reports advocating verifier-guided orchestration and measurement-aware evaluation.

Architectural proposals and prototype demonstrations have established GVR as a practical pattern for reducing stochastic generator errors and for enabling deterministic acceptance criteria. Work advocating generate–verify–repair pipelines argues that interposing mechanistic checks (compilation, tests, policy assertions) converts some classes of stochastic generator failure into mechanically detectable events that can be rejected or repaired [1]. These demonstrations—largely in code-generation and controlled system settings—show GVR can detect structural violations and support iterative repair feedback, but they focus on defect classes that are mechanically decidable. That scope matters: validators that are powerful for syntactic/structural defects may still miss semantic intent misalignment or operator-expectation mismatches that are not encoded in mechanistic constraints.

Benchmarks and task suites in the AIOps/NetOps space document the kinds of operational failures that motivate verifier insertion. Recent benchmark work emphasizes ticket noise, false premises, and wrong-but-plausible outputs that complicate end-to-end automation; controlled suites have been used to measure agent brittleness and the practical costs of erroneous tool calls [2],[3]. These benchmark studies make two points relevant here: first, end-to-end agentic behavior depends heavily on task realism (noisy tickets, ambiguous intent), and second, evaluation protocols and scoring choices (LLM judges, simulators, deterministic validators) materially shape measured success. Importantly, the benchmarks cited do not provide a standardized validator-metric suite tailored to NetOps GVR flows, leaving open how to quantify validator coverage and repair value across operationally representative faults.

System reports and recent preprints advocate tool-augmented orchestrators and verifier-guided recovery as practical mitigations for silent or action-triggering failures in LLM-augmented workflows [4]. These works describe self-healing orchestrators, auditable decision traces, and verifier-feedback loops that in controlled evaluations reduce the incidence of mechanically detectable failures; however, their effectiveness depends on the validators' coverage of relevant defect classes and on orchestration reliability. Surveys of LLM applications in telecommunications and NetOps synthesize adaptation and verification trade-offs, recommending metricized evaluation of verification cost versus correctness and highlighting the need for auditable, evidence-based acceptance criteria [5].

Across these strands the consistent gap is measurement: demonstrations and benchmarks show where deterministic checks help and where generators fail, but none of the cited records supplies an established, empirically validated protocol or metric suite for estimating validator false-positive/false-negative rates, detection latency, or predictive value for operational impact in GVR flows. This gap motivated our preregistered focus on a validator-metric suite and on confirmatory estimation in a paired design. We treat prior work as architectural and motivational evidence—the literature supports the reason to measure validator utility but does not itself provide comprehensive empirical estimates of validator coverage for complex NetOps semantics—so our study was designed to produce artifact-grounded, reproducible estimates under controlled conditions and to surface practical follow-up arms that the archived infrastructure can directly enable.

III. Methodology

Preregistration and primary design. The study preregistration (AC-2026-03) fixed a paired confirmatory design executed via the hosted_netops_gvr_v1 adapter family in scientific_confirmatory mode. The design choice generation_semantics = shared_initial_candidate and initial_generation_calls_per_task = 1 was central: for each deterministic task index the adapter made exactly one initial generator call and that identical generated candidate (the shared initial candidate) was provided to both conditions in the pair. This preregistered semantics guarantees that any observed paired difference (guarded minus baseline) can only arise from downstream deterministic validator behaviour and any permitted repair, not from independent generator sampling differences. The preregistered primary estimand was the paired_success_rate_difference_guarded_minus_baseline. The confirmatory test specified an exact two-sided McNemar test for paired binary outcomes, supplemented by paired bootstrap percentile confidence intervals (bootstrap_resamples = 10000, bootstrap_seed = 1729) as recorded in the analysis artifacts.

Model, sampling, and execution accounting. The declared model in execution/model_configuration.json is "gpt-5-mini" (requested_model = gpt-5-mini). The recorded model configuration also contains operational fields: max_output_tokens = 2000, maximum_attempts_per_call = 1, provider = "openai_responses", store = false, and reasoning_effort = "minimal". Crucially, execution/model_configuration.json records temperature = null (temperature unset). Null/unset is not evidence of deterministic provider-side sampling and does not imply temperature = 0; runtime provider sampling behavior may still be stochastic and is audited via the provider-call records. The execution manifest and deterministic_reconciliation record provider-call audit fields consistent with the preregistered single shared generation per task: hosted_model_call_event_count = 200, completed_hosted_model_call_event_count = 200, cache_hit_count = 0, cache_miss_count = 200, and stage_counts.shared_initial_generation = 200. Because the preregistered model-call budget permitted up to maximum_model_calls = 400, the actual model_calls_used = 200 demonstrates strict adherence to the shared-initial single-sample policy (one hosted-model invocation per task index).

Task generation, constraints, and contamination controls. Tasks were drawn deterministically from deterministic_netops_task_generator_v5 with task_family = intent_configuration_repair_v1 and the preregistered deterministic index-rule challenging_workflow_stress_v1. Each task manifest encodes: initial_state, expected_final_state, an explicit natural-language intent, allowed_fields and allowed_touched_fields, workflow_policies (for example, minimum_active_in_groups, must_be_down_for_fields, protected_interfaces), a required_command_sequence, and a structured difficulty label ("very_hard"/"extreme") that the generator used when instantiating the reference answer and task inputs. To preserve confirmatory integrity the preregistration specified deterministic contamination/exclusion rules: exact-duplicate outputs (identical SHA-256) and near-duplicate tasks (embedding cosine similarity > 0.95) were to be detected automatically and replaced deterministically by the next-index entry to maintain the planned task_count. The analysis artifacts show no contamination exclusions occurred (analysis/contamination_summary.csv empty) and the missingness summary reports complete_pairs = 200, consistent with the preregistered sampling plan.

Validator, scoring rule, and permitted repair policy. The deterministic validator used throughout the study was deterministic_netops_validator_v5 (validator_version = 5.0). The validator implements mechanistic checks that are directly exposed in per-episode validator_trace fields archived for each episode: normalized_configuration (canonical command list), parsed_command_count, required_command_count, observed_touched_field_count, violation_count and violation_codes, valid (boolean), validator_id/version, repair_applied (boolean), and (when applicable) repair_provider_trace metadata. The preregistered binary scoring rule for the confirmatory estimand was full_intent_constraint_validation, which maps directly to validator.valid (true = success, false = failure) in the archived episodes. The pipeline permitted at most one repair call per episode when the validator rejected a candidate; repair events would set repair_applied = true and populate repair_provider_trace fields (these were null for all episodes in this execution). The preregistration also deterministically specified that any repair introducing actions outside the task's allowed_action set would be treated as a policy-violation repair and excluded from the primary success numerator, with that event logged as a policy-violation.

Execution policy, missingness, and sensitivity planning. The preregistered maximum_attempts_per_call = 1 disallowed manual retries; adapter and provider failures were logged and would lead to deterministic exclusions or to classification as technical failures per the missingness plan. Primary confirmatory analysis used only complete pairs; the preregistered sensitivity analysis treated any missing/incomplete episode as a failure (0) for guarded episodes to produce conservative bounds. The execution manifest records completed_episode_count = 400, failed_episode_count = 0, and complete_pair_count = 200; the analysis missingness artifact confirms no observed missing pairs (analysis/missingness_summary.csv). All missingness and audit logs (timestamps, HTTP codes, provider error messages) were archived for reproducibility.

Analysis executor and inferential details. Analysis used the paired_binary_analysis_v1 executor as preregistered. The executor computed the paired 2×2 contingency counts (n_11, n_10, n_01, n_00) from per-episode validator.valid labels; primary testing used the exact McNemar two-sided test appropriate for paired binary outcomes. Confidence intervals for the paired difference were constructed using a paired bootstrap percentile procedure with bootstrap_resamples = 10000 and bootstrap_seed = 1729 (these bootstrap parameters are recorded in analysis/analysis_log.jsonl). Secondary analyses that were preregistered but exploratory (for example, linear models predicting simulated operational-impact proxies from validator metric vectors) were to be treated as exploratory; multiplicity control was only to be applied to exploratory p-values where reported (Benjamini–Hochberg), while point estimates and intervals were to be reported without multiplicity adjustment otherwise.

Deterministic reconciliation and provider audit semantics. The execution bundle includes deterministic_reconciliation artefacts that reconcile task and provider-call accounting with analysis inputs; this reconciliation confirms that both baseline and guarded conditions observed the same shared_initial_candidate for each pair (cross_condition_cache_key_reuse_observed = false indicates no hidden cross-condition cache reuse beyond the single sampled generation). Provider-call audit fields (hosted_model_call_event_count = 200, cache_miss_count = 200) plus the model_calls_used = 200 entry make explicit that one model call produced the shared_initial_candidate per task. These audit fields are central for verifying that the experimental estimand (paired difference under shared_initial_candidate) is correctly identified in the archived artifacts.

Reproducibility, archival artifacts, and permitted reanalyses. All raw responses, per-episode validator traces, provider-call traces, task manifests, model configuration JSON, analysis code, and final numeric artifacts (paired_contingency_table.csv, condition_summary.csv, analysis logs) are archived in the supplied execution bundle referenced elsewhere in this paper. The preregistered artifact set supports direct reanalysis under alternative scoring rules (for example, judging success against a looser parse-based rule), simulated fault-injection applied offline to archived reference answers, or a multi-sample re-run using the same task indices but altering initial_generation_calls_per_task; such follow-up analyses are precisely the artifact-grounded prescriptions we propose and can be preregistered and executed using the same adapter semantics. Important operational note recorded in execution/model_configuration.json: the stored temperature field is null/unset; null does not imply deterministic sampling by hosted providers and therefore does not remove the need to audit provider-call records when assessing generator variance.

IV. Results

Execution accounting and primary outcomes. The adapter completed the run exactly as preregistered: planned_episode_count = 400, completed_episode_count = 400, failed_episode_count = 0, and model_calls_used = 200 (one shared initial generation per pair). Provider-call audit recorded hosted_model_call_event_count = 200, completed_hosted_model_call_event_count = 200, cache_hit_count = 0, cache_miss_count = 200, and stage_counts.shared_initial_generation = 200 (deterministic_reconciliation.json). Contamination and missingness artifacts confirm no exclusions or missing episodes (analysis/contamination_summary.csv empty; analysis/missingness_summary.csv reports complete_pairs = 200 and zeros for all missingness categories).

Primary confirmatory result and exact accounting. The deterministic validator judged every sampled shared initial candidate as valid; no repairs were applied in any episode. Analysis artifacts report baseline_success_count = 200/200 (success_rate = 1.0) and guarded_success_count = 200/200 (success_rate = 1.0). The paired contingency table (analysis/paired_contingency_table.csv) is n_11 = 200, n_10 = 0, n_01 = 0, n_00 = 0. Consequently the paired_success_rate_difference_guarded_minus_baseline = 0.00. The prespecified exact McNemar two-sided test yields p-value = 1.0. The paired bootstrap percentile 95% confidence interval (bootstrap_resamples = 10000, bootstrap_seed = 1729; analysis/analysis_log.jsonl) is [0.00, 0.00]. These numeric artifacts (analysis/condition_summary.csv, analysis/paired_contingency_table.csv, analysis/analysis_log.jsonl) are archived and match the reported values.

Representative per-episode validator diagnostics (artifact-grounded). To illustrate why no repairs were triggered, Table and scoring artifacts contain the validator trace fields for every episode; a compact example (execution/scoring/task-000004-guarded.json) shows the key mechanistic outputs that were recorded for each episode:

pair_id: pair-000004 normalized_configuration: "interface edge2 admin down\ninterface edge2 vlan 40\ninterface edge2 admin up\ninterface edge1 admin down" parsed_command_count: 4 required_command_count: 4 observed_touched_field_count: 3 valid: true repair_applied: false validator_id: deterministic_netops_validator_v5 validator_version: 5.0

Equivalent validator_trace.validation_after.valid = true and repair_applied = false hold for every archived episode; the repair_provider_trace fields are null for all episodes (execution/scoring/*.json entries list repair_provider_trace_path = null). Representative tasks in the artifact_examples (task-000004, task-000005, task-000006) likewise show parsed_command_count equal to required_command_count and valid = true, demonstrating that, for these sampled seeds, generator outputs satisfied the validator's mechanistic constraints exactly.

Secondary and exploratory diagnostics (uninformative under ceiling). Because repair_applied = false for all episodes, secondary metrics that depend on rejection or repair events are degenerate in this execution: per-validator false-positive or false-negative rate estimates cannot be computed from observed rejections; detection-latency and repair-invocation distributions are empty by design. The archived raw_results.jsonl and per-episode validator traces nevertheless provide the full data required to compute these metrics under alternative scoring rules or after applying the follow-up prescriptions (fault injection or multi-sample generation) described elsewhere in this paper.

Artifact-grounded diagnostic interpretation. Under shared_initial_candidate semantics any paired difference could only arise from validator-triggered repair. The degenerate all-valid outcome observed here is explicable from artifact-grounded evidence in three linked facts: (1) the deterministic task generator and the recorded generation seeds produced initial candidates that satisfied the validator's mechanistic intent-constraint checks (as shown by per-episode validation_after summaries); (2) the single-sample-per-task sampling regime removed generator variability as a source of disagreement by design (execution/model_configuration.json and deterministic_reconciliation.json record one shared generation per task); (3) mechanistic validators operate on structural and policy checks and can be blind to semantic intent misalignment (correct-by-structure yet incorrect-by-intent) absent a separate semantic-intent model or operator feedback. These jointly explain why no repairs occurred and why the confirmatory test could not detect the preregistered 10-percentage-point improvement target.

Reproducibility pointers and archived artifacts. All numeric claims above are reproducible from the archived artifacts: execution/raw_results.jsonl (input_results_sha256 = 23f4cd010f3a7419eb21ef9b8caf4dbe7730552eaa3f3eaa 22d2068cf3b5d4e3), execution/model_configuration.json (sha256 = a40e96e2...), analysis/paired_contingency_table.csv (sha256 = 7bc1bd2a...), analysis/condition_summary.csv (sha256 = 1cb072b4...), analysis/analysis_log.jsonl (sha256 = dd2dbf18...), and representative per-episode scoring files (execution/scoring/task-*.json). These files show the per-episode validator.valid flags, repair_applied markers, shared_initial_candidate content and SHA-256s, and provider-call audit entries used to produce the summary statistics above. Because the observed ceiling outcome is entirely recorded in these artifacts, reanalysis under modified scoring rules or preregistered follow-up arms (fault injection, multi-sample generation, multi-validator comparison) can be performed without re-executing the original shared-initial generation stage.

V. Discussion

Design implications of shared_initial_candidate semantics. The preregistered shared_initial_candidate choice isolates downstream validator effects by ensuring both conditions evaluate the same initial candidate for each pair. This isolation is scientifically valuable because any observed paired difference must derive from validator-triggered repair. It also concentrates the risk of a single-sample ceiling: if the generator sample aligns with the validator for all tasks, the design yields zero observed variance in the primary estimand, as occurred here. We emphasize this property because it determines what the paired estimator can and cannot measure: it cannot detect differences attributable to generator sampling or to alternative prompt constraints when the sampling semantics were shared.

Operational interpretation and validator scope. The deterministic validator (deterministic_netops_validator_v5) reliably enforces mechanistic intent-constraint checks (parsed_command_count, required_command_count, violation_codes, valid flag). However, artifact-grounded evidence here shows that correctness-by-structure does not imply semantic intent alignment under operator expectations. That limitation is intrinsic to validators that lack an external intent model or operator feedback; detecting semantic mismatches requires either richer intent models, human-in-the-loop signals, or tailored semantic validators. The present execution does not imply validators are unnecessary—rather, it demonstrates that in a synthetic bank and single-sample regime the validator had nothing mechanical to correct.

Expanded diagnostics grounded in execution artifacts. The archived execution provides explicit numeric diagnostics that clarify why the confirmatory estimator was uninformative: model_calls_used = 200 (one shared generation per pair), complete_pair_count = 200, baseline_success_count = 200, guarded_success_count = 200, and repair_applied = false observed for every episode. The paired contingency is a point mass (n_11 = 200), and the bootstrap procedure (bootstrap_seed = 1729, resamples = 10000) produced a degenerate sampling distribution with 95% percentile CI [0.00, 0.00]. These exact values are available in the analysis artifacts (condition summary and paired contingency outputs) and show that the confirmatory null is not a statistical fluke but a deterministic consequence of the joint generator–validator–sampling configuration used.

Practical lessons for preregistered validator evaluation. From a methodological perspective, two trade-offs are clear and artifact-evident. First, shared_initial_candidate is a powerful control: it guarantees that any observed paired improvement is attributable to downstream validator/repair logic alone. Second, that control increases sensitivity to ceiling effects when generation produces validator-compliant candidates for the chosen seeds. For studies whose primary aim is to measure validator coverage (FP/FN) and repair utility, the preregistered design should therefore include one or both of the following preregistered modifications to avoid degeneracy: (a) a controlled fault-injection arm that deterministically introduces a known fraction f of invalid cases (syntactic, configuration-logic, or semantic-intent swaps) into the evaluation set so that the validator will face realistic defects; (b) a multi-sample generation arm with initial_generation_calls_per_task >= 3 independent draws per task to reintroduce generator variability and produce candidate diversity that can trigger validator rejections and repairs. Both prescriptions were articulated in the archived post-run prescriptions and are directly implementable within the same adapter semantics.

What validator metrics become estimable under those modifications. If faults are injected or multiple independent draws are allowed, the validator-metric suite preregistered in AC-2026-03 becomes estimable: (i) per-validator false-positive rate (FP-rate) as the fraction of validator.accept = true when the task's ground-truth is invalid; (ii) per-validator false-negative rate (FN-rate) as validator.reject = false for candidates that do actually violate the required intent sequence; (iii) mean detection latency measured from model-response receipt to validator decision (records include timestamps in provider- and validator-traces); (iv) repair-invocation rate and post-repair success fraction when repair_applied = true is observed. These metrics map directly to the fields already emitted by the validator in the archived traces (valid, violation_count, repair_applied, parsed_command_count) and to provider-call audit counters; no additional opaque instrumentation is required to compute them once non-degenerate data are present.

Reproducibility and artifact-grounded protocol refinements. The execution artifacts document concrete accounting fields that support exact replication of the intended follow-ups: provider-call audit counts (hosted_model_call_event_count = 200), the model identifier (declared_model_version = gpt-5-mini), the recorded temperature = null/unset (explicitly not evidence of deterministic sampling), and the validator identifier/version (deterministic_netops_validator_v5, v5.0). A follow-up preregistration should (and can, per the archived infrastructure) explicitly set: (1) initial_generation_calls_per_task = k (e.g., k = 3) with independent generation seeds recorded for each draw; (2) a deterministic fault-injection schedule specifying fault classes and the exact fraction f of tasks affected; and (3) a plan to measure detection latency using the existing timestamps. These concrete protocol elements are already supported by the adapter and manifest fields and would remove ambiguity about sampling and allow FP/FN estimation without inventing new instrumentation.

Threats to validity revisited with artifact evidence. Internal validity: the shared_initial_candidate design intentionally removes generator sampling as a confound but introduces a ceiling risk; the exact accounting shows that risk materialized (complete agreement across conditions). Construct validity: validator.valid captures mechanistic constraint satisfaction but not semantic intent alignment—this remains the central construct gap and explains why correct-by-structure outputs can still be operationally unacceptable. External validity: the study executed one model (gpt-5-mini) and one deterministic validator against a synthetic task bank; therefore caution is required when extrapolating to multi-vendor production networks or to models and validators with different coverage. Conclusion validity: the archived bootstrap and contingency outputs (seed/resamples specified) demonstrate the degenerate inferential state (zero variance) and justify treating the confirmatory hypothesis as untestable under these run parameters rather than as a failed hypothesis test.

Operational implications and recommended reporting practices. For experimenters and operators evaluating GVR pipelines, we recommend preregistering both (a) the sampling semantics (shared_initial_candidate vs independent draws) and (b) explicit anti-ceiling design elements (fault injection or multi-draw sampling) whenever the evaluation goal is to estimate validator coverage or repair utility. Empirically reporting the exact counts we used (model_calls_used, complete_pair_count, baseline/guarded success counts, repair_applied counts, bootstrap_seed/resamples) increases interpretability because readers can immediately diagnose whether a noninformative null arose from lack of validator failures or from insufficient sample diversity.

Summary. The execution artifacts and numeric diagnostics make the outcome clear: under the chosen shared_initial_candidate semantics, the combination of deterministic seeds, single-sample generation, and a mechanistic validator that checks structural constraints produced a ceiling effect (200/200 valid). That outcome is scientifically informative about the joint behavior of generator+validator under these settings and usefully constrains what conclusions can and cannot be drawn. It also supplies a concrete, artifact-grounded roadmap—fault injection, multi-sample generation, and multi-validator comparison—that will enable future preregistered studies to estimate the validator metrics of interest and to quantify repair utility in operationally meaningful conditions.

VI. Limitations

- Shared_initial_candidate semantics constrained inference: baseline and guarded reused the same initial candidate per pair; any paired difference could only arise from validator-triggered repair (not from independent generator sampling).
- Synthetic task bank (difficulty 'very_hard'/'extreme') is not equivalent to heterogeneous production networks (multi-vendor devices, real ticket noise); external validity is limited.
- Single-model experiment: only gpt-5-mini was executed; no multi-model generality claims are made.
- No repairs were applied because all initial candidates passed the deterministic validator; therefore the study could not estimate repair effectiveness or validator false-negative behavior in this execution.
- Validator scope: deterministic_netops_validator_v5 enforces mechanistic intent-constraint checks but does not guarantee detection of semantic intent misalignment (correct-by-structure but incorrect-by-intent).
- Recorded execution/model_configuration.json temperature = null/unset does not imply deterministic sampling; provider-side sampling behavior may vary and is documented in execution manifests.

VII. Conclusion

We executed a fully preregistered paired confirmatory study (AC-2026-03) under shared_initial_candidate semantics to measure validator utility in NetOps GVR pipelines. For the synthetic task bank, the sampled gpt-5-mini generations, and deterministic_netops_validator_v5 used here, every sampled initial candidate passed the validator's mechanistic checks so no repairs were applied and guarded success equaled baseline (200/200). The preregistered power assumptions (targeted 10-percentage-point improvement) were invalidated by this ceiling outcome. The result is artifact-grounded and reproducible from the archived bundle. We publish the exact execution and analysis artifacts to support replication and provide concrete, preregistered experiment prescriptions (fault injection, multi-sample generation, multi-validator comparison) that are required to produce estimable validator FP/FN behavior and repair value in operationally meaningful conditions.

Disclosure Statement

Disclosure (concise): Declared model and provider: gpt-5-mini via provider bundle 'openai_responses'. Orchestration/adapter: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5 (validator_version = 5.0). Preregistration: AC-2026-03 (preregistration_sha256 = de37b485421cb0895a98df38b6b3baad1dc7c13c81065e3d 8240d2a94a0ff844). Master prompt immutable reference: provenance/master_prompt.txt, SHA-256 = 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba. Autonomy boundary: a human Research Director selected the broad topic family and approved the master prompt before the autonomy lock; after the lock the AI system autonomously performed literature review, preregistration, experiment design, execution, analysis, internal peer review and manuscript drafting without manual editing of technical content. Sampling/generation semantics: shared_initial_candidate (one initial sampled generation per task reused for baseline and guarded); execution/model_configuration.json recorded temperature = null/unset (null/unset is not evidence of deterministic sampling and does not imply temperature = 0). Call budget and usage (preregistered): maximum_model_calls = 400; actual model_calls_used = 200 (one shared initial generation per pair). All raw outputs, per-episode validator traces, execution manifests, and analysis artifacts are archived in the supplied execution bundle (see execution/execution_manifest.json). No human manually edited the manuscript technical content after the autonomy lock.

References

[1] [1] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN Electronic Journal, 2026. doi:10.2139/ssrn.6754899.

[2] [2] K.-H. Tseng, N. Bogahawatta, Y. Ginige, K. Patel, K. Dakic, S. Seneviratne, "FaulT-Bench: Towards Benchmarking Network Troubleshooting LLM Agents under Unreliable User Tickets," arXiv:2608.27021, 2026.

[3] [3] Y. Liu, C. Pei, L. Xu, B. Chen, M. Sun, Z. Zhang, Y. Sun, S. Zhang, K. Wang, H. Zhang, J. Li, G. Xie, X. Wen, X. Nie, M. Ma, P. Dan, "OpsEval: A Comprehensive IT Operations Benchmark Suite for Large Language Models," arXiv:2310.07637, 2023.

[4] [4] R. S. Babu and A. Agrawal, "Self-Healing Agentic Orchestrators for Reliable Tool-Augmented Large Language Model Systems," arXiv:2606.01416, 2026.

[5] [5] H. Zhou, C. Hu, Y. Yuan, Y. Cui, Y. Jin, C. Chen, H. Wu, D. Yuan, L. Jiang, D. Wu, X. Liu, J. Zhang, X. Wang, J. Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities," IEEE Commun. Surveys & Tutorials, 2024. doi:10.1109/COMST.2024.3465447.